

Arcade OPP

*Documentation Lib
Arcade
2020*

AJOUTER UN JEU

Pour ajouter un nouveau jeu il faut créer une librairie dynamique avec une class héritant de l'interface **IGames.hpp** :

```
#ifndef IGAMES_HPP
#define IGAMES_HPP

#include <iostream>
#include <vector>

#ifndef DISPLAY_T
#define DISPLAY_T

typedef struct display_s {
    std::string ascii;
    std::string foreground;
    std::string background;
    std::string image;
    float x;
    float y;
    bool isPrint;
} display_t;

#endif

namespace Arcade {
    enum statusGame {
        INGAME,
        WIN,
        LOSE,
    };
    class IGames {
    public:
        virtual ~IGames() = default;

        virtual std::vector<display_t> &getInitState() = 0;
        virtual std::vector<display_t> &getState() = 0;
        virtual void move(char input) = 0;
        virtual void restart() = 0;
        virtual uint64_t getScore() = 0;
        virtual Arcade::statusGame &getStatus() = 0;
        virtual bool speedGame() = 0;
    protected:
    private:
    };
}

typedef Arcade::IGames *createGame_t(void);

#endif /* !IGAME_HPP */
```

Le jeu doit utiliser un **vector** de structure **display_t** pour communiquer avec les lib. graphiques

```
std::string ascii;
```

Contient la **valeur** à affiché pour les lib. graphique qui affiche du **texte**.

```
std::string image;
```

Contient le **chemin** vers le **sprite** à affiché pour les lib. graphique qui en utilise.

```
float x;
```

Contient la **position en %** de l'**axe x** de ce qu'il faut afficher (ex : 1.0 = 100%, 0.3 = 30%) par rapport à la fenêtre.

```
float y;
```

Contient la **position en %** de l'**axe y** de ce qu'il faut afficher (ex : 1.0 = 100%, 0.3 = 30%) par rapport à la fenêtre.

```
bool isPrint;
```

Si sa valeur est égal à **true** la librairie affichera l'élément, si il est égal à **false** il ne l'affichera pas.

Les fonctions de votre jeu doivent fonctionner de la manière suivante :

```
std::vector<display_t> &getInitState();
```

doit renvoyer un **vector** de **display_t** qui correspond à l'état **initial** du jeu.

```
std::vector<display_t> &getState();
```

doit renvoyer un **vector** de **display_t** qui correspond à l'état **actuel** du jeu.

```
void move(char input);
```

Est utilisé par le Core du programme **tous les X temps**, l'input donné est sous la forme du char. qui a été pressé par l'utilisateur (si l'utilisateur appui sur z, input = 'z') et input = -1 si l'utilisateur n'appui sur aucune touche

```
void restart();
```

Doit **relancer** le jeu à son état **initial**.

```
uint64_t getScore();
```

Renvoie la valeur **actuelle** du **score** de l'utilisateur.

```
Arcade::statusGame &getStatus();
```

Renvoie l'état **actuel** du jeu, **INGAME** si l'utilisateur est en train d'y jouer, **WIN** s'il a gagné et **LOSE** si il a perdu.

```
bool speedGame();
```

Doit renvoyer **true** si le jeu doit être **accélérer** et **false** si on garde la **vitesse actuelle**.

AJOUTER UNE LIBRAIRIE GRAPHIQUE

Pour ajouter une nouvelle librairie graphique il faut créer une librairie dynamique avec une class héritant de l'interface **IGraphics.hpp** :

```
#ifndef IGRAPHICS_H
#define IGRAPHICS_H

#include <iostream>

#ifndef DISPLAY_T
#define DISPLAY_T

typedef struct display_s {
    std::string ascii;
    std::string foreground;
    std::string background;
    std::string image;
    float x;
    float y;
    bool isPrint;
} display_t;

#endif

namespace Arcade
{
    class IGraphics {
    public:
        virtual ~IGraphics() = default;

        virtual char getEvent(void) = 0;
        virtual void print(display_t &display) = 0;
        virtual void destructor() = 0;
        virtual void refresh() = 0;

        virtual int getWindowHeight() = 0;
        virtual int getWindowWidth() = 0;
        virtual bool clockDisplay() = 0;
    };
}

typedef Arcade::IGraphics *createLib_t(void);

#endif /* !IGRAPHICS_H */
```

Les fonctions de votre librairie graphique doivent fonctionner de la manière suivante :

```
char getEvent(void);
```

Renvoie en format ascii la touche qui est appuyée par l'utilisateur.

```
void print(display_t &display);
```

Affiche l'élément **display_t** donné sur la **fenêtre**.

```
void refresh();
```

Actualise l'affichage de la fenêtre.

```
int getWindowHeight();
```

Renvoie la **hauteur** de la **fenêtre** en **pixel**.

```
int getWindowWidth();
```

Renvoie la **largeur** de la **fenêtre** en **pixel**.

```
bool clockDisplay();
```

Renvoie **true** si la lib. peut utiliser `print()` et sinon **false** (permet de limiter les fps).